SDN-enabled Quantized LQR for Smart Traffic Light Controller to Optimize Congestion

ANUJ SACHAN and NEETESH KUMAR, Indian Institute of Technology-Roorkee, Roorkee, India

Existing intersection management systems, in urban cities, lack in meeting the current requirements of self-configuration, lightweight computing, and software-defined control, which are necessarily required for congested road-lane networks. To satisfy these requirements, this work proposes effective, scalable, multi-input and multi-output, and congestion prevention-enabled intersection management system utilizing a software-defined control interface that not only regularly monitors the traffic to prevent congestion for minimizing queue length and waiting time but also offers a computationally efficient solution in real-time. For effective intersection management, a modified linear-quadratic regulator, i.e., Quantized Linear Quadratic Regulator (QLQR), is designed along with Software-defined Networking (SDN)-enabled control interface to maximize throughput and vehicles speed and minimize queue length and waiting time at the intersection. Experimental results prove that the proposed SDN-QLQR improves the comparative performance in the interval of 24.94%–49.07%, 35.78%–68.86%, 36.67%–59.08%, and 29.94%–57.87% for various performance metrics, i.e., average queue length, average waiting time, throughput, and average speed, respectively.

CCS Concepts: • **Networks** → **Programmable networks**; • **Computer systems organization** → **Sensors and actuators;**

Additional Key Words and Phrases: Intelligent transportation system (ITS), smart city, traffic light controller (TLC), urban traffic network

ACM Reference Format:
Anuj Sachan and Neetesh Kumar. 2024. SDN-enabled Quantized LQR for Smart Traffic Light Controller to Optimize Congestion. *ACM Trans. Internet Technol.* 24, 1, Article 7 (February 2024), 25 pages. https://doi.org/10.1145/3641104

1 INTRODUCTION

Traffic light control system design has been a key research domain in the **Intelligent Transportation System (ITS)**, considering the advances in **Artificial Intelligence (AI)**, **Software-defined Networking (SDN)**, communication (5G/6G) and computation (Cloud/Fog/Edge computing) technologies, and smart city infrastructure [1]. Several traffic light control methods were proposed by the researchers considering different scenarios. In general, control policies are broadly divided into two major categories: (i) fixed-time policy and (ii) traffic responsive policy [40]. The historical traffic demands within the signalized urban area determine the duration of the phase in fixed-time

This research is supported by the Science and Engineering Research Board (SERB), Department of Science and Technology, Government of India (Grant No. SER-EEQ/2019/000182), and the Council of Scientific and Industrial Research (CSIR), New Delhi, Government of India (Grant No. 22-0817-19-EMR-II)..
Authors' address: A. Sachan and N. Kumar, Department of Computer Science and Engineering, Indian Institute of Technology-Roorkee, Roorkee, Uttarakhand, India, 247667; e-mails: anuj_s@cs.iitr.ac.in, neetesh@cs.iitr.ac.in.
Publication rights licensed to ACM. ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only. Request permissions from owner/author(s).
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1533-5399/2024/02-ART7
https://doi.org/10.1145/3641104

policy. In spite of the fact that fixed-time policy has a fixed value for the phase duration, it is not able to handle the dynamics of real-time traffic situations. Addressing the limitations of fixed-time control policy, traffic responsive traffic light control policy is used to optimize signal timing based on real-time traffic situations. In practice, actuated traffic light signal control is the most common and effective method for controlling traffic-responsive signals [42]. Once a vehicle is detected, actuated control increases the phase time by the amount of travel time until the maximum allowed green phase duration is achieved [40].

Traditionally used centralized architectures lack in scalability and suffer from a single point of failure, which requires maintaining a backup that is an expensive operation in non-virtualized infrastructure [7]. To overcome this, decentralized architectures are explored [8]. **Decentralized Architecture (DA)** offers a scalable architecture but has high operational, communication, maintenance, and management costs due to a lack of programmable control and synchronization overheads [8]. Further, DA experiences a significant delay in the collaborative decision-making process due to synchronization overhead for large-scale deployments [6]. Hence, decentralized or centralized control architectures are not sufficient enough to provide a manageable, scalable, and cost-effective solution. To overcome the limitations of these traditional architectures, a new paradigm named SDN architectures is utilized as the network archetype in vehicular communication networks that offer global coverage, low maintenance, programmable control interface, virtual infrastructure deployment, and auto-configurable features [29, 31]. In addition, SDN-based architectures are also deployed as hybrid architectures to utilize the complementary advantages of both types of architectures [25]. Recently, a **Linear Quadratic Regulator (LQR)** was introduced to minimize the traffic delay based on the traffic control system model [41, 43]. However, the limitations of the existing state-of-the-art, including LQR for the intersection management systems, are observed as follows: (1) Existing intersection management systems are based on decentralized or centralized approaches; distributed controllers have higher maintenance and management costs due to lack of auto-configuration and programmable control features, and centralized controllers are lack in scalability due to single point of failure. (2) LQR-based approach has proven effective for traffic light control, however, LQR-based controllers manage the intersection locally, and splatted control signals only into fixed proportional intervals this may cause starvation condition if priority and heterogeneity among the vehicles are accounted. (3) The current LQR-based intersection management systems route traffic based on local traffic decisions and do not consider traffic load imbalances, which results in traffic jams in certain areas of the city. To overcome the above mentioned limitations, this study suggests an SDN-enabled improved version of LQR, i.e., **Quantized LQR (SDN-QLQR)** intersection management system, and the notable contributions of the SDN-QLQR are itemized as follows:

— To overcome the limitations of the centralized and decentralized architectures, an SDN-based architecture (Figure 1) is proposed, in which an SDN controller (at control plane) collects real-time vehicular dynamics, i.e., speed, waiting time, throughput, and so on, from each intersection (lies at data plane), and it balances the traffic-flow and prevents the congestion in the city.
— To account the heterogeneity, a prioritized weight assignment method is proposed for each type of vehicles. With this, for traffic light control decision-making, lane prioritized weight is accounted in contrast of vehicle count used in state-of-the-art methods, and the effectiveness of the same is rigorously analysed through a set of experiments.
— To generate the traffic light control signals at each intersection locally, a QLQR is proposed, generating the control signals (R/Y/G) by taking the feedback from the SDN controller. The

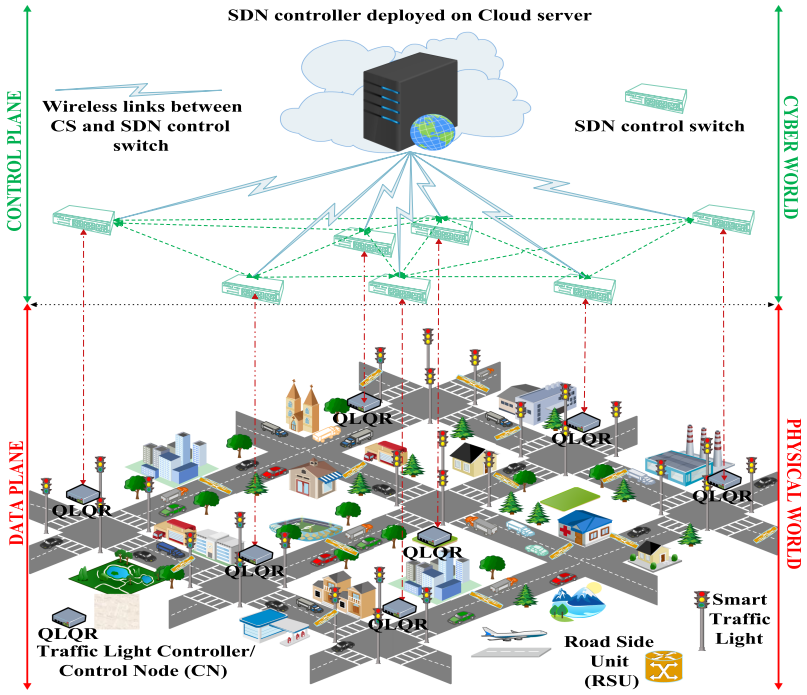


Fig. 1. Cyber-physical world for SDN-enabled ITS.

QLQR generates the control signals in the form of quantum to overcome the starvation condition.

— To prevent congestion and balance the traffic, the SDN module deploys an algorithm over the Cloud that monitors the vehicular traffic periodically, and based on the vehicular dynamics and traffic conditions, it calculates a threshold value to determine the congestion. Based on the threshold value, it generates congestion prevention signals to the respective intersections.

— To evaluate the performance of SDN-QLQR, a simulation study is developed on the Indian city OpenStreetMap using the **Simulation of Urban Mobility (SUMO)** simulator [17]. The proposed SDN-QLQR is compared with the state-of-the-art methods, i.e., LQR [38], **max-pressure (MP)** [24], **dynamic traffic light controller (DTLC)** [22], and **self-organizing traffic lights (SOTL)** [9]. Subsequently, the SDN-QLQR is also compared with other SDN-based TLC methods, i.e., **SDN utilized deep reinforcement learning (SDN-DRL)** [20], and **SDN-enabled quantized max-pressure controller (SDN-QMPC)** [29], and the obtained results prove its effectiveness.

The article is organized as follows: The related works are introduced in Section 2 in the field of the intersection management system. In Section 3, the problem statement and solution approach are presented. In Section 4, the proposed SDN-QLQR traffic light control framework is described. In Section 5, the baseline methods and simulation-based setup are described. An experimental analysis of the proposed work is presented in Section 6, considering different vehicles' arrival rates, comparison with same-line state-of-the-art methods, and also comparison with other SDN-based TLC methods. Finally, the conclusions and suggestions for future work are presented in Section 7.

2 RELATED WORK

Varaiya [37] proposed the MP-driven traffic light control policy, which needs queue lengths' estimation for the road lanes. The significant advantages of MP-driven controller are: (i) traffic estimation is distributed over junctions following distributed approach; (ii) advance knowledge regarding traffic demand is not required; and (iii) balance is there as long as the network is under certain presumptions can be established below any control. Subsequently, Barman et al. [4] presented a cyclic MP controller to determine the total cycle length when vehicular traffic increases. The main disadvantage of this work is that it does not consider the first-in-first-out traffic pattern. Cools et al. [9] proposed a traffic signal control method utilizing SOTL. This method was effective in comparison to traditional fixed-time controllers. Few efforts have also been made to develop traffic signal control methods using neural networks [5, 35], some of them are based on fuzzy logic-based control [15, 30], some of them are based on reinforcement learning [19, 26], and some of them are based on deep reinforcement learning-based control [28, 39]. These models were proposed to optimise traffic light control signals based on various parameters, such as traffic delay, throughput, average speed, energy consumption, and so on [16]. As a result of their complexity, high-performance variability, and specific hardware requirements, these systems are not widely adopted in real-time implementation using light-computing-based IoT devices.

Some of the most popularly used traffic-responsive signal control systems for network-level traffic management are: (i) **Sydney Coordinated Adaptive Traffic System (SCATS)** [32], (ii) **Split Cycle Offset Optimization Technique (SCOOT)** [14], and (iii) **Traffic-responsive Urban Control (TUC)** [18]. Both SCATS and SCOOT integrate a network model that takes real-time traffic data as the input and calculates the total number of vehicle stops as the output. TUC [21] uses LQR theory to model urban network traffic. In addition, Hong Wang et al. [38] also proposed an adaptive linear-quadratic regulator utilizing calibrated VISSIM model. This model identifies undetermined system dynamics, and it can handle system uncertainties caused by traffic patterns and environmental fluctuations. However, this algorithm suffers from the problem of fixed cycle length duration of the regulator and starvation [3]. To overcome the issues, this study suggests a SDN-QLQR-based traffic-light controller and introduces the concept of time-quantum along with LQR. The proposed QLQR has the following attributes: (i) low processing complexity; (ii) low communication and computation overhead; (iii) controlling traffic signals with precision and effectiveness; and (iv) low deployment cost. Further, the SDN component deals with congested intersections with an effective programmable interface.

3 PROBLEM STATEMENT AND SOLUTION APPROACH

This study identifies a problem of dynamic traffic light scheduling by considering the heterogeneity among the vehicles, congested regions, and imbalance distribution of vehicular traffic on the road lanes. The same problem is solved by proposing a SDN-enabled quantized linear quadratic regulator-based model named as SDN-QLQR. The identified problem becomes complex by considering real-time communication, heterogeneity among vehicles, dense network of intersections, frequent uncertain vehicular dynamics, and unexpected congestion. The pictorial visualization of the proposed architecture is depicted in Figure 1. Generally, it works in two major steps; first, the relevant vehicular traffic parameters, i.e., vehicle type, count, ID, speed, waiting time, and so on, are collected through the vehicular network [11]. To do this, it is assumed that RSUs/OBUs, RFID sensors, and so on, are deployed in the environment, which is shown in Figure 1. The proposed QLQR-based algorithmic model is deployed on Raspberry Pi/Nvidia Jetson (edge computing) device to compute the received raw data received via sensors to obtain vehicular traffic state and other relevant traffic parameters, such as, vehicle speed, queue length, waiting time, as examined in

Reference [36]. By using this input, in step 1, the QLQR model makes traffic light control decisions about R/G/Y phase duration. All the intersections are assumed to be equipped with the QLQR-based algorithmic module, and these modules may act in parallel at all the intersections. QLQR is based on a LQR model; the LQR is opted due to its significant advantages over other control strategies in respect of control design: (i) LQR is basically an automated way of finding an appropriate state-feedback controller; (ii) LQR is robust to model unexpected disturbances, uncertainties, and errors; (iii) LQR is a fundamental optimal control strategy, which can be adaptive when combined with system parameters estimation. The proposed QLQR produces the traffic light control signals in the form of quantum, offering a fair chance to each road lane and avoiding starvation.

In the second step, an SDN-enabled vehicular network is established. The SDN controller periodically interacts with the intersections to monitor the congestion level in the city. If a congested intersection is identified based on the proposed threshold policy, then the SDN controller sends a congestion prevention signal to the respective intersection to maximize outflow and minimize inflow. Otherwise, the traffic flow is managed by following the traffic light control signals produced by the QLQR module. The SDN controller is located on the cloud server, as depicted in Figure 1.

Commonly, traffic light controllers generate three control signals: **yellow (Y)**, **green (G)**, and **red (R)**. A single traffic light is unable to manage traffic flow from various directions, so multiple traffic lights must be deployed at intersections and work in synchronization. The traffic lights must be timed at intersections to maintain non-conflicting flow from multiple directions. The time interval for which a signal (R/Y/G) stays on is called the phase duration. At each intersection, all phases must be changed cyclically, which is known as a cycle and repeats continuously. No vehicles are allowed to pass through during the red signal phase, whereas vehicles in the corresponding lane have the right of way during the green signal phase. The goal of this research is to generate traffic light control signals that consider the relatively high congestion regions in urban cities and balance traffic flow while optimizing several quality of service parameters, such as throughput, speed, queue length, and waiting time at each intersection, by accounting for the assigned prioritized lane load based on vehicle heterogeneity.

4 THE PROPOSED SDN-QLQR MODEL

4.1 System Model

This sub-section models the set of road lanes and the set of intersections in the city utilizing graph $G(J, E)$. The vertices represent the set of intersections in the urban network where roads meet $J \in \mathbb{R}_{\geq 0}$ ($\mathbb{R}_{\geq 0}$ is a set of nonnegative real numbers). The edges E represent the road lanes' set represented by the ordered pair $(p, q) \in E$. Vehicles enter the intersection at input nodes, which are denoted by $I_{(p,q)}$. The output nodes are the ones that are given by $O_{(p,q)}$, at which vehicles leave the intersection. The graph constructed is a directed graph.

A **Cell Transmission Model (CTM)** is utilized to describe the queue length in each lane. It is a discrete-time state-space formulation of the relationship between crucial traffic flow parameters. The queue length is determined by the total number of vehicles in a lane at any time and is represented as $QL(t) \in \mathbb{Z}_{\geq 0}$. Where $\mathbb{Z}_{\geq 0}$ is a set of nonnegative integers. The traffic inflow is determined by the total number of vehicles approaching the intersection in a given time duration and is represented as $IF(t) \in \mathbb{R}_{\geq 0}$. The traffic outflow is determined by the total number of vehicles leaving the intersection in a given time and is represented as $OF(t) \in \mathbb{R}_{\geq 0}$.

The traffic outflow $OF^J_{O(p,q)}(t)$ can be computed as follows:

$$OF^J_{O(p,q)}(t) = \min\left(QL^J_{(p,q)}(t), MOF^J_{(p,q)}\left(\lambda^J_{(p,q)}(t)\right)\right), \tag{1}$$

where $MOF^{J}_{(p,q)}(\lambda^{J}_{(p,q)}(t))$ is the maximum traffic outflow rate of lane (p, q) at intersection J given the green phase duration $\lambda_{(p,q)}$ at time t. The maximum outflow rate $MOF^{J}_{(p,q)}(\lambda^{J}_{(p,q)}(t))$ is determined by the traffic light signal and the lane capacity.

The traffic inflow $IF^{J}_{I_{(p,q)}}(t)$ can be computed as follows:

$$IF^{J}_{I(p,q)}(t) = \sum_{(p',q')\in I_{(p,q)}} OF^{J'}_{O(p',q')}(t), \tag{2}$$

where $I_{(p,q)}$ is the set of roads (p', q') that enter road (p, q) at junction J, and $OF^{J'}_{O(p',q')}(t)$ is the traffic outflow of road (p', q') at junction J'. The queue length $QL_{(p,q)}(t + 1)$ can be obtained by the following first-order difference equation:

$$QL_{(p,q)}(t + 1) = QL_{(p,q)}(t) + H - P, \tag{3}$$

$$H = \sum_{c=1}^{n} Weight_{IF_c}(t), P = \sum_{d=1}^{m} Weight_{OF_d}(t),$$

where $\sum_{c=1}^{n} Weight_{IF_c}$ is the sum of the weights of vehicles entering the queue during the time t, and $\sum_{d=1}^{m} Weight_{OF_d}$ is the sum of the weights of vehicles leaving the queue during the time t.

Subsequently, the **Average Queue Length (*AQL*)** at any intersection is computed as

$$AQL_{junction}(t + 1) = \sum_{k\in n} \left(\frac{QL_{(p,q)_k}(t + 1)}{N_k} \right), \tag{4}$$

where n denotes number of intersecting roads at the intersection, and N denotes vehicles' total number. Following this, the waiting time $WT_{(p,q)}(t + 1)$ of the vehicles is updated as

$$WT_{(p,q)}(t + 1) = \frac{[QL_{(p,q)}(t) \times LaneLength]}{[IF_{I_{(p,q)}}(t) - OF_{O_{(p,q)}}(t)]}. \tag{5}$$

After that, the **Average Waiting Time (*AWT*)** at any intersection is calculated as follows:

$$AWT_{junction}(t + 1) = \sum_{k\in n} \left(\frac{WT_{(p,q)_k}(t + 1)}{N_k} \right). \tag{6}$$

The obtained AQL and AWT are computed based on this system model. These parameters are utilized as a state to the QLQR and SDN controller. Following this, QLQR generated the phase duration. The generated phase duration is used to update the traffic light phase.

4.2 Logical Flow

Figure 2 shows the logical flow of the proposed SDN-QLQR. It starts with real-time vehicular dynamics as inputs. From this input, the *Real-time Traffic Parameter Extractor* module extracts useful traffic parameters such as **Vehicle Identity (*VID*)**, **Average Speed (*AS*)**, vehicle type, and so on. Following this, using the *Lane-wise Weight Calculator* module ($Weight_{lane}$), the Average Waiting Time of the road lane ($AWT_{roadlane}$) and Average Queue Length of the road lane ($AQL_{roadlane}$) parameters are calculated, considering the heterogeneity among the vehicles. These parameters are sent to the SDN and QLQR components at the same time.

In the QLQR component, the *QLQR*() module calculates linearized system parameter matrices ($A(t_j)$ and $B(t_j)$) at the jth junction and the state feedback gain matrix (K_j) using Equations (8) and (9). After that, the quantum value $u(t_j)$ is calculated by *Sum of Diag*($K(t_j)$). If $u(t_j)$ is equal to 0, then it calls Equation (10); otherwise, it calls Equation (11). At the same time, $u(t_j)$ is forwarded to the **Set Phase Duration (SPD)** module.

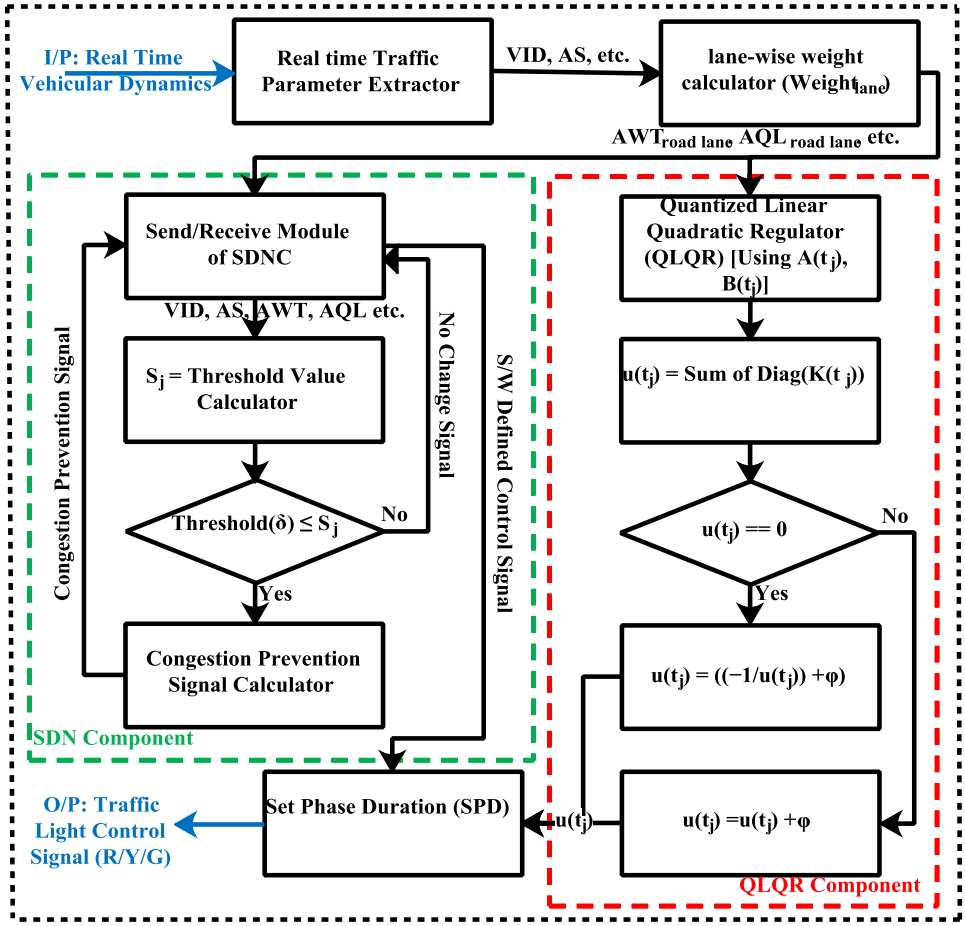


Fig. 2. Proposed SDN-QLQR logical flow.

Simultaneously, the SDN component initiates one of its modules, *Threshold Value Calculator*, which calculates the threshold (S_j), to prevent congestion and balance traffic in the city. Researchers in Reference [2] suggest that average queue length, speed, and delay should be used as the primary parameters in quantifying congestion. Therefore, all three parameters are accounted for to estimate the congestion value. If S_j is greater than the pre-defined threshold (δ), then it invokes the *Congestion Prevention Signal Calculator* module to generate the *Congestion Prevention Signal*; otherwise, it produces a no-change signal. The same signal is sent to the message send/receive module to manage the traffic flow accordingly.

The *Congestion Prevention Signal* includes the new phase duration for the corresponding intersection as well as its neighboring intersections. All the message communication between the control plane and the data plane is handled by the *send/receive Module of SDNC*. Utilizing the data plane, the control plane receives queue length, waiting time, and speed information. Based on this information, the control plane calculates the congestion value and determines whether the *Congestion Prevention Signal* needs to be sent to the SPD module of the traffic light control. If the congestion value is above a pre-defined threshold, then the control plane sends the *Congestion Prevention Signal* to the SPD module, which adjusts the traffic light phase duration based on

the quantum value, and the new phase duration is received in the signal. Following this, the SPD module generates the traffic light control signal (R/Y/G) as the output.

4.3 Algorithm Description

Algorithm 1: QLQR starts by taking the inputs as appeared in Algorithm 1. It can be seen that before going to algorithmic procedure, $Q = Q^T > 0$ and $R = R^T > 0$ are taken as the identity matrices (I) with the corresponding dimensions, where Q is the state-cost weighting matrix, and R is the input-cost weighting matrix. In line 1, j is the loop variable ($0 \leq j \leq J-1$). Next, in line 2, time stamp for jth junction, i.e., t_j is initialized to 0, and road lane variable k is also initialized to 0. Next, T_j is fixed to 60 s that is the cycle phase duration for jth junction. Next, if $(t_j \% T_j) == 0$, QLQR enters in its execution loop for the maximum number of simulation steps. After that, in line 6, Vehicle Identity, i.e., $VID_j(t_j)$ at jth junction at time stamp t_j is calculated using $getVehiclelist(N_j)$, where N_j is the total number of vehicles at jth junction. Before going to further steps, in line 7, Weight Matrix, i.e., $Wmatrix_j(t_j)$ is calculated using $WeightCalculation(N_j, VID_j)$ at time stamp t_j, the definition for the same is given in Algorithm 2. Next, for kth road lane AQL and AWT are calculated using Equations (4) and (6), respectively. Here, k is the loop variable, which is used to represent the number of lanes at the jth junction ($0 \leq k \leq n_{j-1}$), and n_j is the total number of road lanes at the jth junction. In line 9, AQL for kth road lane at jth junction is calculated using Equation (4). After that, $AQL_{j,k}$ is then inserted into the diagonal of matrix B that is visualized as

$$B(t_j) = \begin{bmatrix} AQL_{j,0} & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & AQL_{j,k} \end{bmatrix}.$$

After that, in line 11, AWT is calculated as using Equation (6).

After that, $AWT_{j,k}$ is then inserted into the diagonal of matrix A as follows:

$$A(t_j) = \begin{bmatrix} AWT_{j,0} & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & AWT_{j,k} \end{bmatrix}.$$

Now, the system parameter matrices $A(t_j)$ and $B(t_j)$ are estimated as per lines 9–12 at each time stamp (t_j). Following this in line 13, the pressure for kth road lane of jth junction ($X_{j,k}$) based on AQL and AWT, is calculated as follows:

$$X_{j,k} = \chi \times AQL_{j,k}(t_j) + \zeta \times AWT_{j,k}(t_j), \tag{7}$$

where χ and ζ are queue length and waiting time factors, respectively. Following this, the state feedback gain matrix $K(t_j)$ is obtained by solving the following non-linear differential equation of first order, i.e., Riccati equation [33], for matrices $S(t_j) = S^T(t_j) > 0$ and $K(t_j)$ simultaneously:

$$\mathbf{Q} + \mathbf{A}^T(t_j)\mathbf{S}(t_j)\mathbf{A}(t_j) - \mathbf{S}(t_j) - \mathbf{A}^T(t_j)\mathbf{S}(t_j)\mathbf{B}(t_j)\mathbf{K}(t_j) = 0, \tag{8}$$

$$K(t_j) = \mathbf{B}^T(t_j)\mathbf{S}(t_j)\mathbf{B}(t_j) + \mathbf{R}^{-1}\mathbf{B}^T(t_j)\mathbf{S}(t_j)\mathbf{A}(t_j). \tag{9}$$

Now, the quantized time $u(t_j)$ is calculated by adding the all diagonal elements of a matrix $K(t_j)$. If $u(t_j) == 0$, then

$$u(t_j) = u(t_j) + \phi, \tag{10}$$

where an extra value, i.e., ϕ, is added as a correction factor. The value of this factor can be adjusted based on the architecture of each individual junction. Else,

$$u(t_j) = (-1/u(t_j)) + \phi. \tag{11}$$

ALGORITHM 1: QLQR

Input: Total Number of junctions (J), Total Number of Vehicles (N), Average Speed (AS), Vehicular Network of the city.
Output: Traffic light control signal (Green/Yellow/Red).
Notation:
t_j: Time Stamp for jth junction
n_j: Number of roads intersecting at jth junction
$u(t_j)$: Quantized time
$Wmatrix_j$: Weight Matrix for jth junction
$A\&B$: Linearized System parameter matrices(control input)
ind : Lane index
Initialize:
Q = $IdentityMatrix(I)$ ▷ State-Cost weighting matrix
R = $IdentityMatrix(I)$ ▷ Input-Cost weighting matrix

for each $j \in J$ **do in parallel**
  Initialize: $j = 0, k = 0, t_j = 0$;
  $T_j = 60$ ▷ Initial phase duration for jth junction
  while $t_j < SimulationSteps$ **do**
    if $(t_j \% T_j) == 0$ **then**
      $VID_j(t_j) = getVehiclelist(N_j)$;
      $Wmatrix_j(t_j) = WeightCalculation(N_j, VID_j)$;
      for $k \in n_j$ **do**
        Calculate $\mathbf{AQL_{j,k}(t_j)}$ using Equation (4);
        $B(t_j).DiagAppend(AQL_{j,k})$;
        Calculate $\mathbf{AWT_{j,k}(t_j)}$ using Equation (6);
        $A(t_j).DiagAppend(AWT_{j,k})$;
        Calculate $\mathbf{X_{j,k}}$ using Equation (7);
      end for
      Calculate $\mathbf{K(t_j)}$ using Equations (8) and (9);
      $u(t_j) = SumOfDiagK(t_j)$;
      if $u(t_j) == 0$ **then**
        Calculate $\mathbf{u(t_j)}$ using Equation (10);
      else
        Calculate $\mathbf{u(t_j)}$ using Equation (11);
      end if
      $[X_m, ind]$ =max$_{1 \le n \le k}$ $(X_{j,k})$;
      $SetPhaseDuration(u(t_j), X_{j,k}, X_m, ind, n_j, T_j)$;
      SndMsgToCS(AWT_j, AQL_j, AS_j, T_j);
      T_j = SDNC (AWT_j, AQL_j, AS_j, T_j);
      RecvMsgFromCS(T_j);
    end if
    $t_j = t_j + 1$;
  end while
end for

Following this, in line 22, the maximum pressure lane (X_m) and respective lane index (ind) are estimated. Following this, in line 23, traffic light phase is updated by setting phase duration using $SetPhaseDuration(u(t_j), X_{j,k}, X_m, ind, n_j, T_j)$, the definition for the same is given in Algorithm 3. Next, traffic parameters are sent to **control server (CS)** using $SndMsgToCS(AWT_j, AQL_j, AS_j, T_j)$, based on these parameters, SDN-Controller calculates new T_j that is received by the jth junction, the definition for the same is given in Algorithm 4. After that t_j is incremented by one. The process is repeated for the number of simulation steps.

ALGORITHM 2: Weight Calculation

Input: N_j, VID_j
Output: $Wmatrix_j(t_j)$
Notation:
$W_f = [W_p, W_b, W_t, W_a, W_{fb}, W_o]$ ▷ Weight factor

1: Initialize: $Wmatrix_j = [], i = 0$;
2: **for** $i \in N_j$ **do**
3: $l_i = vehicle.getTypeID(VID_{i,j})$
4: **if** $l_i ==' Police'$ **then**
5: $Wmatrix_j.append(W_p)$;
6: **else if** $l_i ==' Bus'$ **then**
7: $Wmatrix_j.append(W_b)$;
8: **else if** $l_i ==' Truck'$ **then**
9: $Wmatrix_j.append(W_t)$;
10: **else if** $l_i ==' Ambulance'$ **then**
11: $Wmatrix_j.append(W_a)$;
12: **else if** $l_i ==' Fire_brigade'$ **then**
13: $Wmatrix_j.append(W_{fb})$;
14: **else**
15: $Wmatrix_j.append(W_o)$;
16: **end if**
17: **end for**
18: **return** $Wmatrix_j(t_j)$

Algorithm 2: The Weight Matrix($Wmatrix_j(t_j)$) corresponding to the vehicles at the junction is generated by the function $WeightCalculation(N_j, VID_j)$. Before entering into for loop, $Wmatrix_j$ is initialized to $empty$, and i is initialized to 0, where i is the loop variable ($0 \leq j \leq N_j - 1$), N_j is the number of vehicles at the jth junction. After entering into for loop, l_i receives vehicle type by using $vehicle.getTypeID(VID_{i,j})$ function corresponding to vehicle ID, i.e., $VID_{i,j}$. Next, with the help of If-$Else$ conditions, the priority weight of each vehicle category is estimated and appended to the weight matrix, i.e., $Wmatrix_j(t_j)$ utilizing vehicle-specific weight factor, where $W_{fb}, W_t, W_b, W_a, W_p$, and W_o represent the weight factors of Fire_brigade, Truck, Bus, Ambulance, Police, and other types of vehicles, respectively. With this, the $Wmatrix_j(t_j)$ is generated.

Algorithm 3: $SetPhaseDuration(u(t_j), X_{j,k}, X_m, ind, n_j, T_j)$ is utilized to update/set the traffic light control signal phase (Green/Yellow/Red) duration. Here, k is the loop variable ($0 \leq k \leq n_j - 1$) to represent the number of road lanes at the jth junction, and where n_j is the total number of road lanes at the jth junction. In line 4, total number of quantum slots are counted for kth lane for jth junction and stored in $QT_{Count}[k]$). Using the assertions, each lane's traffic signal phase (Green/Yellow/Red) duration is proportionally updated/set. For the maximum pressure lane,

ALGORITHM 3: SetPhaseDuration

Input: $u(t_j), X_{j,k}, X_m, ind, n_j, T_j$
Output: Green/Yellow/Red Phase Duration

1: *Intialize:* $QT_{Count}[k] = 0$;
2: **while** $(sum(QT_{Count}[k])! = 0)$ **do**
3: **for** $k \in n_j$ **do**
4: $QT_{Count}[k] = \frac{\frac{X_{j,k} \times T_j}{\sum_{i \in N_{j,k}} X_{j,k}}}{u(t_j)}$;
5: **if** $QT_{Count}[k] == 0$ **then**
6: *continue*;
7: **end if**
8: **if** $k == ind$ **then**
9: $SetGreenSignalPhase(laneID_k, 2 \times u(t_j))$;
10: $SetRedSignalPhase(laneID_l, 2 \times u(t_j))$; ▷ *where*, $1 \leq l \leq n_j$, and $l \neq k$
11: $QT_{Count}[k] = QT_{Count}[k] - 2$;
12: **else**
13: $SetGreenSignalPhase(laneID_k, u(t_j))$;
14: $SetRedSignalPhase(laneID_l, u(t_j))$; ▷ *where*, $1 \leq l \leq n_j$, and $l \neq k$
15: $QT_{Count}[k] = QT_{Count}[k] - 1$;
16: **end if**
17: $SetYellowSignalPhase(laneID_{all}, 3)$;
18: **end for**
19: **end while**

two quantums are allotted, and remaining other lanes, one quantum slot is allotted following an iterative approach. In Algorithm 3, the detailed pseudo-code is given. Before switching from one phase to the next (red/green), the duration of the yellow phase is set to 3 s, which indicates phase switching is about to occur. This iterative procedure stops only when the sum of all the quantum slot counts becomes zero.

Algorithm 4: SDN controller starts its execution by receiving the traffic parameters, i.e., AWT_j, AQL_j, AS_j, and T_j from jth junction. Constants are initialized in lines 1 to 4, but their values can be modified according to the specific requirements of the application. These values are taken from our previous work [20], in which we do a detailed study to decide these values. Next, for the jth junction, $NAWT_j$, $NAQL_j$, and NAS_j are calculated, where $NAWT_j$, $NAQL_j$, and NAS_j are the normalized value of average waiting time, average queue length, and average speed, respectively. Following these parameters, S_j is calculated as follows:

$$S_j = \alpha \times NAWT_j + \beta \times NAQL_j - \gamma \times NAS_j, \tag{12}$$

where α, β, and γ are waiting time, queue length, and speed factors, respectively. If $S_j \geq \delta$, then T_j is increased by 25% of T_j, and T_l for its lth neighbouring junction is decreased by 25% of T_j, where $0 \leq l \leq |Neighbours| - 1$. Finally, respective T_j is sent back to jth junction in which it executes.

4.4 LQR and QLQR Theoretical Analysis on Underlying Assumptions

To conduct a theoretical analysis of the proposed QLQR method, each of the fundamental components and conditions are explained as follows:

ALGORITHM 4: SDN Controller(SDNC)

Input: AWT_j, AQL_j, AS_j, T_j
Output: Updated phase duration for the Junction (T_j).

$\alpha \leftarrow 0.4$; ▷ Waiting time factor
$\delta \leftarrow 0.6$; ▷ Expected congestion threshold value
$\gamma \leftarrow 0.3$; ▷ Speed factor
$\beta \leftarrow 0.3$; ▷ Queue length factor
for $j \in J$ **do**
RecvMsgFromController(AWT_j, AQL_j, AS_j);
end for
for each $j \in J$ **do in parallel**
    $NAWT_j = \frac{AWT_j - AWT_{min}}{AWT_{max} - AWT_{min}}$;
    $NAQL_j = \frac{AQL_j - AQL_{min}}{AQL_{max} - AQL_{min}}$;
    $NAS_j = \frac{AS_j - AS_{min}}{AWT_{max} - AS_{min}}$;
    Calculate $\mathbf{S_j}$ using Equation (8);
    if $\delta \leq S_j$ **then**
        $T_j = T_j + 0.25 \times T_j$;
    end if
    for $l \in Neighbour_j$ **do**
        $T_l = T_l - 0.25 \times T_j$;
    end for
    SndMsgToController(T_j);
end for

— The system is represented by the following state-space equations [34], commonly in continuous time:

$$\dot{x}(t) = Ax(t) + Bu(t).$$

Here, $x(t)$ is the state vector, and $u(t)$ is the control input. A and B are the linearized system parameter matrices.

— The cost function [34] is defined as follows:

$$\Im = \int_0^\infty (x^T Q x + u^T R u) dt.$$

Here, both the Q and R matrices are set to identity matrices to provide equal importance to states and control inputs.

— The LQR control law [27] is given by

$$u(t) = -K(t)x(t),$$

where $K(t)$ is the state feedback gain matrix, calculated by solving the following algebraic Riccati equation [33]:

$$Q(t) + A^T(t)S(t)A(t) - S(t) - A^T(t)S(t)B(t)K(t) = 0,$$

$$K(t) = B^T(t)S(t)B(t) + R^{-1}(t)B^T(t)S(t)A(t),$$

where both matrices $A(t)$ and $B(t)$ are diagonal matrices with non-negative diagonal elements (either zero or positive), which means that the eigenvalues of $A(t)$ and $B(t)$ are either zero or have positive real parts. In our case, $A(t)$ and $B(t)$ matrices contain the values of

Table 1. Scenario of Four-way Intersection

Lane	Vehicle	Vehicle Type	Vehicle Weight
East	E1	Police	5
	E2	Bus	6
	E3	Car	4
	E4	Truck	3
	E5	Ambulance	10
North	N1	Police	5
	N2	Truck	3
	N3	Bus	6
West	W1	Car	4
	W2	Car	4
	W3	Fire_brigade	10
	W4	Truck	3
South	S1	Ambulance	10
	S2	Ambulance	10
	S3	Fire_brigade	10
	S4	Bus	6

AWT and AQL, and these are non-negative values. Further, a symmetric matrix $S(t)$ is utilized by following the assumption $S(t) = S^T(t) > 0$, which ensures that matrix $S(t)$ is positive definite [10].

Assumption: The positive definiteness of matrix $S(t)$ in the Riccati equation ensures that the system is stabilizable.
Proof: It is already established that eigenvalues of A, A^T, B, and B^T are either zero or have positive real parts. Thus, the presence of $S(t) > 0$, in the $A^T(t)S(t)A(t)$ and $A^T(t)S(t)B(t)K(t)$ ensures that the output of these is positive definite in the algebraic Riccati equation. Further, the presence of $S(t) > 0$ in the expression for $K(t)$ also contributes to the positive definiteness. Therefore, the matrix $S(t)$ in the Riccati equation is taken as positive definite, ensuring the system's stability.

Following this, the LQR established its stability based on the assumptions that we followed. The QLQR represents an improved version of the LQR, ensuring that any assumptions made to create the QLQR do not violate the fundamental principles of LQR. Therefore, these assumptions also hold the mathematical validity of QLQR control strategies for the generation of quantized time.

4.5 An Example to Understand the System

To better understand the approach of the proposed model, let us consider a scenario at a four-way intersection. The initial cycle phase duration (T) is set to 60 s. In the East lane, there are five vehicles: E1, E2, E3, E4, and E5; in the North lane, there are three vehicles: N1, N2, and N3; in the West lane, there are four vehicles: W1, W2, W3, and W4; and in the South lane, there are four vehicles: S1, S2, S3, and S4. Table 1 depicts the vehicle types and their corresponding weight factors. The behaviour of the SDN-QLQR is highly dependent on the priority weights assigned to vehicles, as it is designed for a heterogeneous scenario. The vehicle weight assignment is advised as per the priority of the vehicle, considering a typically localized scenario. Now, it can be observed that the queue length in terms of vehicle weight for the East lane is 28, for the North lane is 14, for the West lane is 21, and for the South lane is 36. Assuming a waiting time of 1 s for all vehicles

in all lanes. Now the pressure of each road lane is calculated using Equation (7). The pressure of the East, North, West, and South lanes is 8.8, 4.6, 6.7, and 11.2, respectively. Further, we estimate the A and B matrices using the queue length and waiting time. The LQR method generates the quantized time by using Equations (8)–(11). The approximately quantized time for the East, North, West, and South lanes is 8.03, 4.07, 7.04, and 12.02 s, respectively. Using this quantized time and the lane with the maximum pressure, Algorithm 3 determines the phases for each lane. First, the South lane receives a green signal phase duration of approximately 24.04 s, during which S1, S2, S3, and S4 will be allowed to pass. Simultaneously, the red signal phase duration for all other lanes is 24.04 s, meaning no vehicles are allowed to pass for the next 24.04 s. Similarly, all the lanes will follow the same. Additionally, the information is periodically sent to Algorithm 4, which is used to generate congestion prevention signals for the corresponding intersection. The same procedure is repeated until all the vehicles have passed through the intersection.

5 BASELINE METHODS AND SIMULATION-BASED SETUP

This section provides a brief overview of the baseline state-of-the-art methods used for the comparative study.

5.1 Baseline Methods

5.1.1 Linear Quadratic Regulator. LQR [38] control is a new approach that can significantly minimize the congestion at the intersection by improving the signal timing. LQR control is a feedback control strategy that utilizes a linear quadratic cost function to reduce the system's control and state effort. Real-time traffic information is used to modify the LQR controller, enabling it to adjust to shifting traffic circumstances. Therefore, LQR control is adjusted to the phase duration to improve the traffic flow at the intersection. In this way, this work presents a new LQR control method for optimizing traffic signals in urban traffic networks. The major challenge of this work is that it does not consider heterogeneity among the vehicles and has a fixed cycle length duration.

5.1.2 Max-pressure. A max-pressure traffic control system is a decentralized control algorithm to optimize vehicular traffic flow at signalized intersections [24]. In this work, the performance of max-pressure controller on a simulated traffic network. In this work, authors found that controller significantly minimized the total travel time in the network compared to the fixed-time controller. The max-pressure controller also improved the vehicle's throughput and travel time at intersections.

5.1.3 Dynamic Traffic Light Controller. DTLC [22] is a traffic light control method that does not consider vehicle heterogeneity and has a fixed cycle length duration. There are three take asides to deploy DTLC: flow, WT, and random. In the flow-based approach, the light turns green based on the number of vehicles entering the intersection. In the WT approach, the light turns green for the lanes with the longest wait times. In the random approach, each lane has a dynamic controller that records the number of vehicles that have passed through during the current red light phase. When the number of vehicles exceeds a threshold, the light turns green for that lane and the counter is reset to zero.

5.1.4 Self-Organizing Traffic Lights. SOTL [9] is a method that is used to control traffic lights without coordination among multiple intersections. Each traffic light generates the traffic signal considering only the local traffic conditions. This work has the potential to be more efficient and adaptable than traditional traffic light control methods, which are generally based on fixed-phase scenarios. The researchers found that utilizing SOTL decreases congestion and improves the traffic

flow at the intersection. The major disadvantage of this method is that it only considers a single intersection and cannot handle unexpected changes in traffic flow at any intersection.

5.2 SDN-based TLC

5.2.1 SDN Utilized Deep Reinforcement Learning. DRL [20] is a machine learning technique used to train agents to learn optimal policies in complex environments like congestion management in urban areas. This work used a DRL agent to minimize congestion and improve traffic flow. The authors first collected data, such as number of vehicles on each road, traffic speed, and traffic density. This data was used to train a DRL agent to generate the efficient phase duration. After that, the action of the DRL agent was used in SDN, and deployed on control server, to generate the congestion prevention signal at the intersection. The significant challenges of this work [20] are: DRL is computationally expensive to train and also sensitive to the quality of the training data.

5.2.2 SDN-enabled Quantized Max-pressure Controller. SDN-QMPC [29] is a method used to generate traffic light phase durations in the form of dynamic time quantum by accounting the heterogeneity among vehicles. This work found that the quantized version of the max-pressure algorithm significantly reduced the total average waiting time and queue length in the vehicular network compared to the max-pressure algorithms. This method also improved the average speed and throughput of the vehicles at intersections.

6 EXPERIMENTAL STUDY

The effectiveness of SDN-QLQR is analyzed using the simulation prepared in SUMO [17]. With this simulation, the proposed SDN-QLQR modules are implemented in Python programming language. The performance behavior of the SDN-QLQR is evaluated using several performance metrics, i.e., AQL, **ThroughPut (TP)**, AWT, and AS. Additionally, QLQR is compared to other state-of-the-art approaches of similar nature, i.e., LQR [38], DTLC [22], MP [24], and SOTL[9] to validate its performance.

6.1 Simulation Parameters and Methodology

To implement SDN-QLQR, Python was utilized to code all the modules, and a vehicular network simulation was established using SUMO. A map of Gwalior city from OpenStreet is obtained for download as a .osm file and converted into a network of road lanes and traffic light intersections using SUMO inbuilt tool, i.e., net-convert. To manipulate the traffic behavior in a SUMO simulation using Python, SUMO offers a module known as the **Traffic Control Interface (TraCI)**. Using TraCI, Python can interface with SUMO, enabling traffic flow control in SUMO through algorithmic code written in Python. The SUMO simulation parameters are listed in Table 2. The binomial distribution with parameter μ, which denotes the maximum number of simultaneous arrivals, is utilized to generate the vehicles in the SUMO inbuilt "Randomtrip.py" Python script by giving the argument to –binomial. The value of μ is set to one to carry out this work for generating vehicles on the network, which can be changed per the application's requirements.

6.2 Experimental Results Analysis

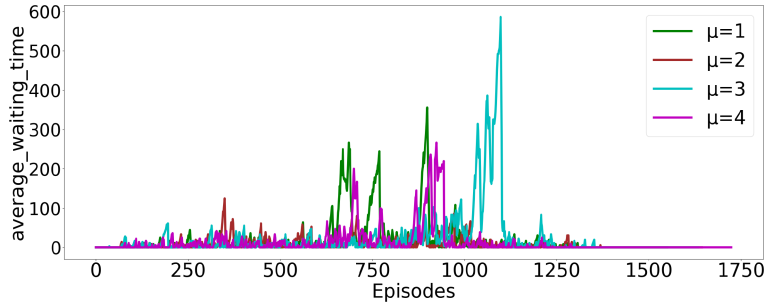
6.2.1 SDN-QLQR Performance Behaviour Study on Different Vehicle's Arrival Rates. The behavior of the proposed SDN-QLQR is analyzed by conducting a series of experiments on various arrival rate values (μ). The performance behavior is analyzed through several performance metrics (AWT, AQL, TP, and AS). Figures 3(a)–3(d) show the results for the performance metrics AWT, AQL, TP, and AS, respectively. The X-axis represents episodes (number of simulation rounds), while the Y-axis represents the corresponding performance metric. In Figures 3, μ = 1 indicates the arrival rate

Table 2. SUMO Simulation Traffic Parameters

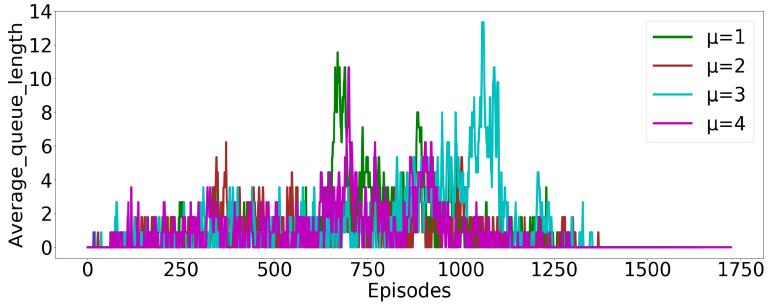
SUMO Simulation Parameters	Value
Length of Car	5 m
Length of Police vehicle/Ambulance	7 m
Length of Bus/Truck/Fire_brigade	10 m
Length of bike	4 m
Other Vehicle's length	4.5 m
Max. Lane Speed	22.0 m/s
Max. Speed of Car/Police vehicle	22.22 m/s
Max. Speed of Ambulance	27.78 m/s
Max. Speed of Bus/Truck/Fire_brigade	16.67 m/s
Max. Speed of bike	22.22 m/s
Other Vehicle's Max Speed	13.9 m/s
Max. Acceleration of all vehicles	0.8 m/s^2
Max. De-acceleration of all Vehicles	4.5 m/s^2
Driving Proficiency(σ)	0.5
Detector Frequency	60
Min. Gap between Two Vehicles	2.5 m
Maximum Number of Lanes	4
Minimum Number of Lanes	2
Road lane type	Unidirectional
Number of Traffic Light Junctions	16
Latitude of the map	26.161674
Longitude of the map	78.072584

(arrival/insertion) of vehicles (one/period), $\mu = 2$ indicates the arrival rate of vehicles (two/period), $\mu = 3$ indicates the arrival rate of vehicles (three/period), and $\mu = 4$ indicates the arrival rate of vehicles (four/period). Figure 3 shows that for very low arrival rates ($\mu = 1$), both AWT and AQL are minimized, which is expected. However, SDN-QLQR successfully maintains its performance for higher arrival rates ($\mu = 2$, $\mu = 3$, and $\mu = 4$), which is the primary objective of this work. Additionally, the behavior of SDN-QLQR for AS depicts a corresponding behaviour, which is prudent. Further, TP remains consistently high across all the scenarios. In summary, the results demonstrate that the performance of SDN-QLQR remains usual despite increasing arrival rates. Hence, results of Figures 3 verify the effectiveness of the SDN-QLQR during peak traffic hours. The reason for this performance improvement is that the proposed SDN-QLQR is implemented considering various traffic dynamics, and the SDN control server continuously monitors the traffic congestion and takes real-time actions accordingly.

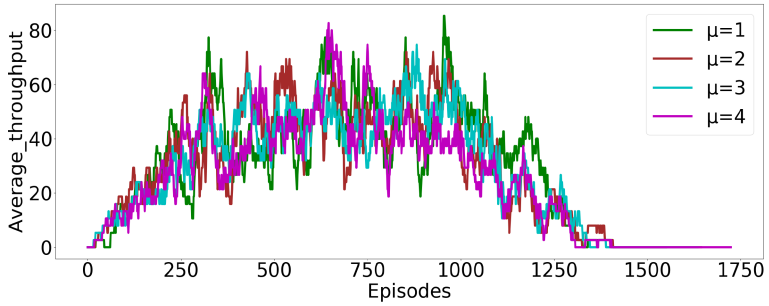
6.2.2 SDN-QLQR Comparative Analysis with QLQR and LQR. SDN-QLQR behaviour is also analysed by conducting a set of experiments on different versions of LQR. In Figure 4, legends represent the different versions of LQR, i.e., SDN-QLQR, QLQR, and LQR. Figures 4(a)–4(d) show the results corresponding to AWT, TP, AQL, and AS performance metrics, respectively, where episodes are shown on X-axis, and the performance metric is shown on Y-axis. From the Figure 4(a), it is analyzed that SDN-QLQR minimizes AWT w.r.t. QLQR, and LQR by approximately 18.52% and 24.94%, respectively. One of the most important performance metrics to determine the effectiveness of the model, is the AWT. Similarly, from Figure 4(b), it is observed that SDN-QLQR minimizes AQL by approximately 31.13%, and 35.78% in comparison to QLQR and LQR, respectively. In a similar manner, Figure 4(c) corresponds to the performance improvements of SDN-QLQR for TP, SDN-QLQR



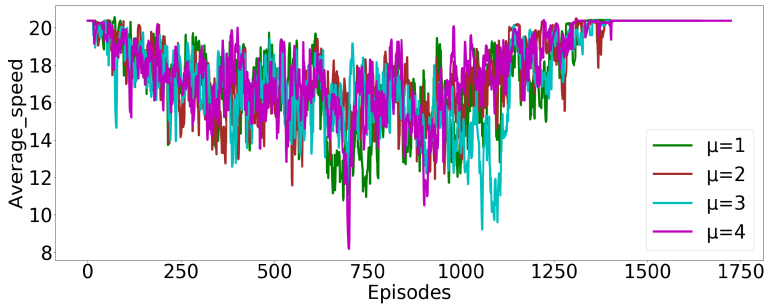
(a) Results on average waiting time



(b) Results on average queue length

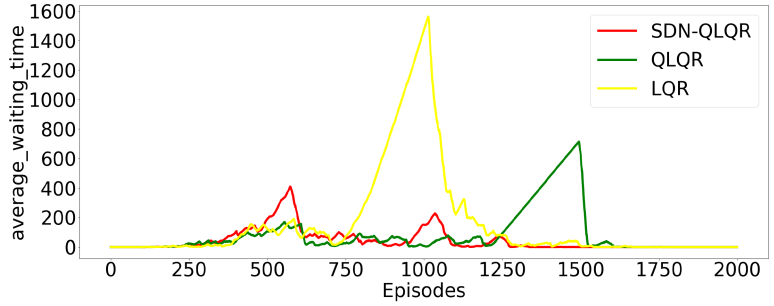


(c) Results on throughput

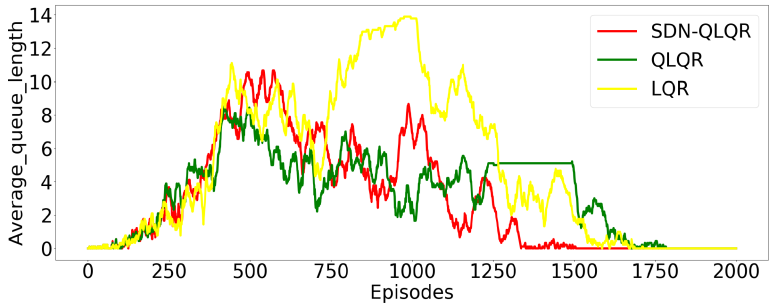


(d) Results on average speed

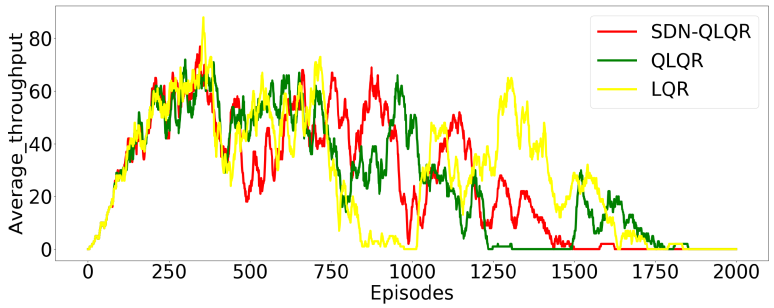
Fig. 3. Comparative results of SDN-QLQR on different arrival rates of vehicles.



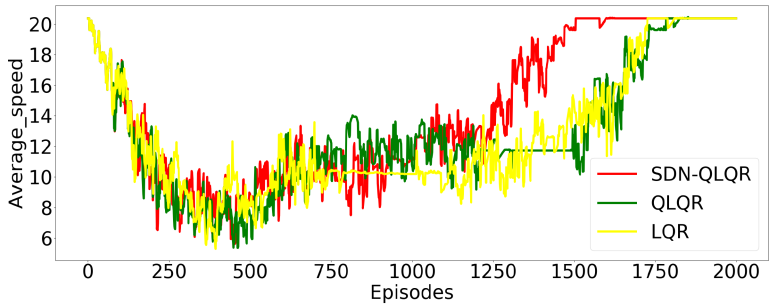
(a) Results on average waiting time



(b) Results on average queue length



(c) Results on throughput



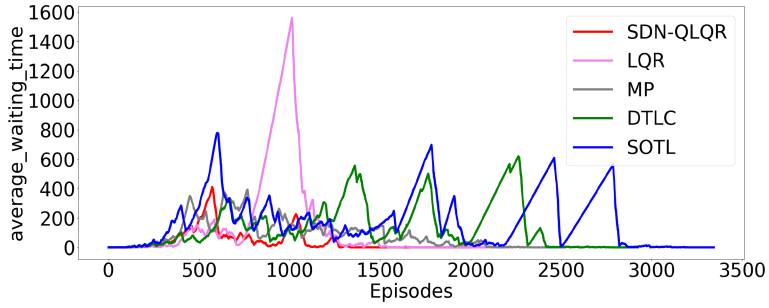
(d) Results on average speed

Fig. 4. Comparative results of SDN-QLQR on QLQR and LQR.

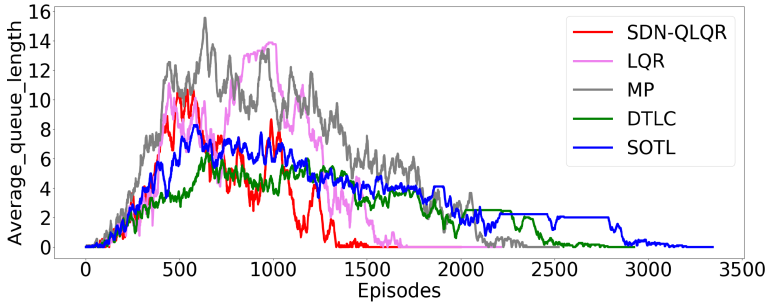
improves 28.93%, and 36.67% performance over QLQR and LQR, respectively. Finally, Figure 4(d) depicts the comparative analysis of the SDN-QLQR for the *AS* performance metric. From Figure 4(d), it is observed that SDN-QLQR beats QLQR and LQR in terms of *AS* by 21.36% and 29.94%, respectively. It is also observed that SDN-QLQR reaches the maximum speed value first in comparison to QLQR and LQR. Based on the analysis of the results shown in Figure 4, it is noticed that the traffic flow is smoother throughout the simulation episodes. In brief, we can say that the proposed SDN-QLQR model significantly improves the performance on different versions of LQR due to inclusion of both quantized and SDN components; quantized component takes care of an effective phase duration distribution for all lanes at the junction, and SDN component generate congestion prevention signal to minimize the congestion by monitoring the and identifying the congested junction.

The total simulation time depends on simulation settings such as the number of vehicles, vehicle routes, and so on. For all the experiments and state-of-the-art methods, we have the same configuration. The proposed SDN-QLQR algorithm takes approximately 39.12 s to complete the simulation, while QLQR and LQR take 57.38 and 63.03 s, respectively. The SDN-QLQR significantly reduces the total simulation time compared to QLQR and LQR while considering various performance metrics (AQL, AWT, TP, and AS). The SDN-QLQR improves this simulation time due to its algorithmic performance in optimizing several performance metrics. Since, in the simulation, all the vehicles have predefined routes to reach their destinations, and vehicles are self-destroyed once they reach their respective destinations. Simulation continues if any vehicle is live or has not reached its destination. With this reason, SDN-QLQR improves the computation time in comparison to LQR and QLQR approaches.

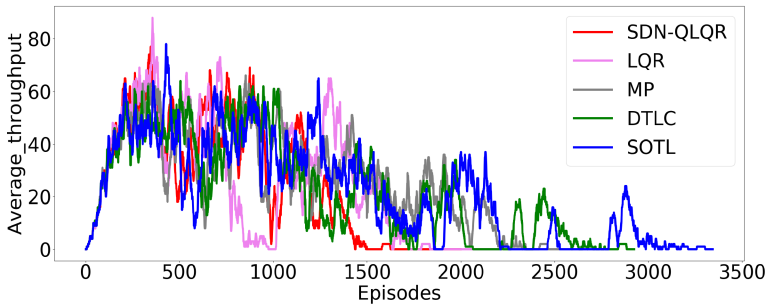
6.2.3 SDN-QLQR Comparative Study with other State-of-the-Art Methods. This sub-section is dedicated to analyze the comparative performance behaviour of the SDN-QLQR with several state-of-the-art methods, i.e., LQR [38], MP [24], DTLC [22], and SOTL [9]. Figure 5(a) presents the comparative results of SDN-QLQR with other same-line state-of-the-art methods to analyze the performance improvements on the *AWT* performance metric. From the results, it is analyzed that SDN-QLQR minimizes *AWT* w.r.t. LQR, MP, DTLC, and SOTL by approximately 24.94%, 31.79%, 37.27%, and 49.07%, respectively. Figure 5(b) shows the comparative results corresponding to the *AQL* performance metric. From the results shown in Figure 5(b), it is analyzed that SDN-QLQR minimizes *AQL* by 35.78%, 49.76%, 54.71%, and 68.86% in comparison to LQR, MP, DTLC, and SOTL methods, respectively. Using the identical set of experiments, Figure 5(c) presents the comparative results for the *TP* performance metric. The comparative results state that the SDN-QLQR maximizes the *TP* by 36.67%, 49.67%, 55.33%, and 59.08% in comparison to LQR, MP, DTLC, and SOTL, respectively. Finally, Figure 5(d) depicts the comparative analysis of the SDN-QLQR for the *AS* performance metric. The results in Figure 5(d) dictate that the SDN-QLQR beats LQR, MP, DTLC, and SOTL in terms of *AS* by 29.94%, 42.80%, 47.67%, and 57.87% performance improvement, respectively. In Figure 5(d), even if a high difference among the five models is not clearly noticeable, on careful inspection, it can be inferred that SDN-QLQR reaches its maximum *AS* value first than others; it indicates that it has the capability to alleviate traffic congestion sooner as compared to other models. In other words, it can be stated that the sooner traffic is cleared by the model or achieved high speed, the model is comparatively better than other models. In brief, from the comparative study, it is analyzed that the SDN-QLQR model significantly improves performance on different performance metrics. The reason for the same is that the QLQR module uses an effective time quantum to improve the phase duration among all the lanes at the intersection by accounting the heterogeneity among the vehicles. Additionally, the SDN module regularly monitors the congestion and generates the congestion prevention signal to avoid highly congested routes or traffic jams.



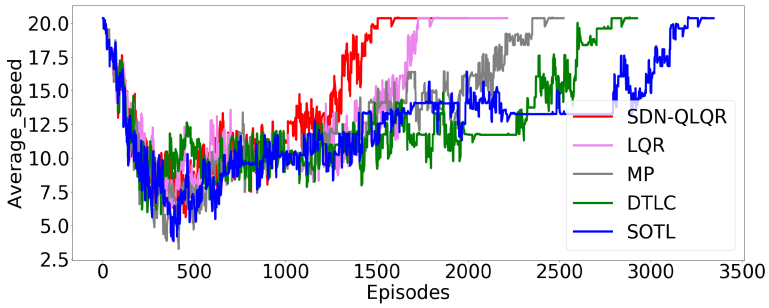
(a) Results on average waiting time



(b) Results on average queue length



(c) Results on throughput



(d) Results on average speed

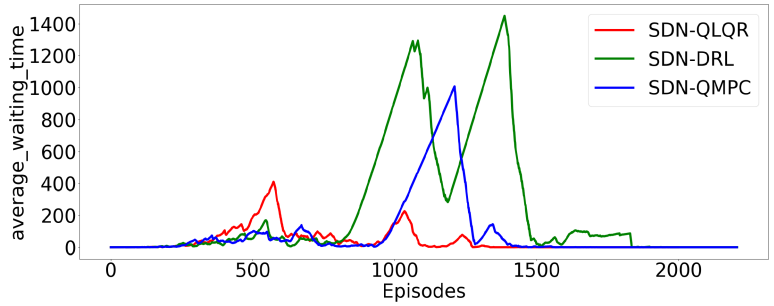
Fig. 5. Comparative results of SDN-QLQR with other algorithms.

6.2.4 SDN-QLQR Comparative Study with other SDN-based TLC Methods. This sub-section is dedicated to analyze the comparative performance behaviour of the SDN-QLQR on various performance metrics with other SDN-based TLC methods, i.e., SDN-DRL [20] and SDN-QMPC [29]. Figure 6(a) represents the comparative results of SDN-QLQR with other SDN-based TLC methods to analyze the performance improvements on the *AWT* performance metric. The results show that SDN-QLQR minimizes *AWT* w.r.t. SDN-DRL and SDN-QMPC by approximately 26.58% and 28.65%, respectively. Figure 6(b) shows the comparative results corresponding to the *AQL* performance metric. From the results shown in Figure 6(b), it is analyzed that SDN-QLQR minimizes *AQL* by 44.48%, and 46.98% in comparison to SDN-DRL and SDN-QMPC methods, respectively. Figure 6(c) presents the comparative results for the *TP* performance metric. The comparative results state that the SDN-QLQR maximizes the *TP* by 43.69% and 47.13% in comparison to SDN-DRL and SDN-QMPC, respectively. Finally, Figure 6(d) depicts the comparative analysis of the SDN-QLQR for the *AS* performance metric. The results in Figure 6(d) dictate that the SDN-QLQR beats SDN-DRL and SDN-QMPC in terms of *AS* by 36.52% and 39.18% performance improvement, respectively. In brief, it is analyzed that the SDN-QLQR model significantly improves the performance on different performance metrics with other SDN-based TLC methods. The reason for the same is that the SDN-QLQR method generates an efficient time quantum to improve the corresponding intersection's control signals, considering the vehicles' heterogeneity. Simultaneously, the SDN module continuously monitors traffic congestion and generates signals to prevent congestion.

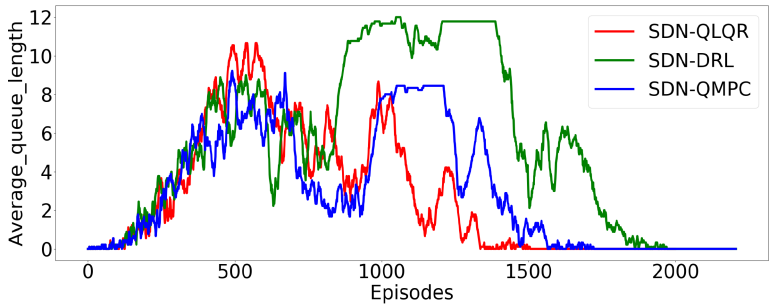
6.3 Analysis and Discussion of Latency in an SDN-based Architecture

As discussed, SUMO [17] is used to generate real-time traffic. However, to analyze the feasibility of field implementation, a realistic test-bed setup is developed in our laboratory as depicted in Figure 7. This test-bed setup is composed of three different components: (i) SUMO simulator running on a system with a 64-bit PC with a Core i7-9700 processor operating at 3.00 GHz and 16 GB RAM, (ii) traffic light controller running on a microprocessor (Raspberry Pi 3 Model B+ [12] with a 1.4 GHz 64-bit quad-core processor and 4 GB RAM), and (iii) SDN controller running on a server with VM (a 64-bit Core Xenon Silver 4215 processor operating at 2.5 GHz and 32 GB RAM, VM (Oracle VirtualBox) is configured with 8 GB RAM, 80 GB storage, 8 core processor with Ubuntu 20.04.6 LTS, OpenFlow protocol [23], running with an external virtual switch). In the SUMO simulation, virtual sensors, acting as loop detectors, are used to detect vehicle presence on the road, extracting data such as speed, occupancy, and count. The SUMO simulation transmits this information via the TCP/IP protocol to the TLC, which is implemented on the Raspberry Pi. The TLC, in turn, calculates the quantized time to generate the traffic light control signals and periodically communicates the same traffic states to the SDN controller. The SDN controller receives the traffic states and generates congestion-prevention signals for the intersection.

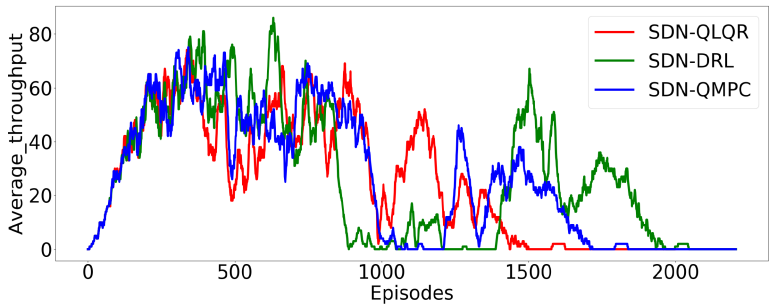
In this study, all communication has occurred within our institute's network with an average bandwidth of 71 Mbps. The data transmitted from SUMO to TLC or TLC to the SDN (or vice-versa) controller consists of numerical values in the form of text data. Based on the above setup an experiment was run, from the experiment,noted the following: (i) average latency from SUMO simulator to TLC is bound to 13 ms, (ii) average latency from TLC to SDN controller (or vice-versa) is bound to 35 ms. The observed latency is quite acceptable in real-world implementation scenarios [13]. Further, in the context of this study, the real-time communication between TLC and the SDN controller is periodic, and the period of communication is at every phase duration cycle. The cycle phase duration (T) is set to a minimum of 16 s for a four-way intersection and 12 s for a three-way intersection. As mentioned in the algorithm, when the time quantum ($u(t)$) is zero, ϕ estimates the time quantum according to Equation (10), with a consistent value of 4 s for all experiments. Therefore, the minimum cycle phase duration (T) remains 12 and 16 s for three-way



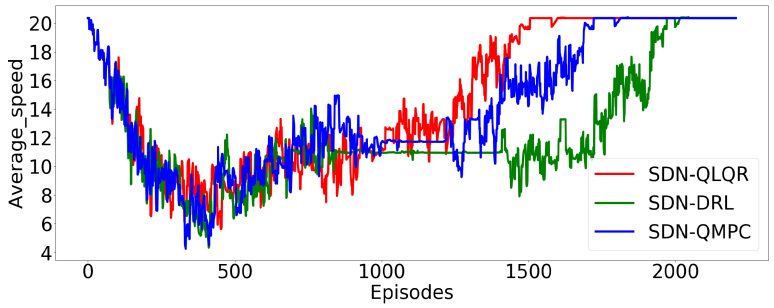
(a) Results on average waiting time



(b) Results on average queue length



(c) Results on throughput



(d) Results on average speed

Fig. 6. Comparative results of SDN-QLQR with other SDN-based TLC methods.



Fig. 7. Test-bed setup.

and four-way intersections, respectively. In summary, the SDN controller and TLC communication will occur after T. This establishes that the proposed solution has significant potential for field implementation.

7 CONCLUSION AND DISCUSSION

In this work, a novel QLQR-based intersection management system utilizing a software-defined control interface is proposed. The QLQR component is used to generate the signal for controlling traffic lights at the junction, and SDN component is used to deal with the congestion by generating congestion prevention signal. The SDN-QLQR is validated via a realistic simulation, that is designed utilizing SUMO [17] tool by importing an OpenSteetMap for the Indian city. To validate the comparative performance of the SDN-QLQR, the outcomes of SDN-QLQR are compared with several state-of-the-art approaches, i.e., LQR [38], MP [24], DTLC [22], and SOTL [9] considering several performance metrics. The SDN-QLQR is also compared with other SDN-based TLC methods, i.e., SDN-DRL [20], and SDN-QMPC [29]. The comparative outcomes serve as supporting evidence for the efficacy of the proposed SDN-QLQR Algorithm. In future work, with a novel policy,will explore the different traffic pattern scenarios, such as the impact of traffic accidents, road closures, and others.

The focus of this study is limited to the implementation of SDN-QLQR in traffic light control decision-making. The investigation currently does not encompass an analysis of communication overhead and latency in real-world field implementations. Therefore, our future research endeavors aim to establish a practical testbed for SDN-QLQR, leveraging the OpenFlow protocol as well as wireless communication protocols like IEEE 802.11p and potentially Cellular communication 5G or 6G. Specifically, this testbed prototype will target addressing significant challenges related to creating the hardware test environment, including aspects such as real-time traffic data capture, deployment expenses, network stability, communication latency, and other pertinent factors.

REFERENCES

[1] Mahdi Abbasi, Ali Najafi, Milad Rafiee, Mohammad R. Khosravi, Varun G. Menon, and Ghulam Muhammad. 2020. Efficient flow processing in 5G-envisioned SDN-based internet of vehicles using GPUs. *IEEE Trans. Intell. Transp. Syst.* 22, 8 (2020), 1–10. https://doi.org/10.1109/TITS.2020.3038250

[2] Md Aftabuzzaman. 2007. Measuring traffic congestion—A critical review. In *Proceedings of the 30th Australian Transportation Research Forum.*

[3] P. G. Balaji, X. German, and Dipti Srinivasan. 2010. Urban traffic signal control using reinforcement learning agents. *IET Intell. Transp. Syst.* 4, 3 (2010), 177–188.

[4] Simanta Barman and Michael W. Levin. 2022. Performance evaluation of modified cyclic max-pressure controlled intersections in realistic corridors. *Transportation Research Record* 2676, 6 (2022), 110–128.

[5] Nisha Singh Chauhan and Neetesh Kumar. 2022. Traffic flow forecasting using attention enabled Bi-LSTM and GRU hybrid model. In *Proceedings of the International Conference on Neural Information Processing*. Springer, 505–517.

[6] Zhiwen Chen, Qiong Hao, Hong Huang, and Cheng Qiao. 2023. Delay-tolerant distributed algorithms for decision-making in vehicular networks. *Asia-Pacific Journal of Operational Research* 40, 4 (2023), 2340004.

[7] Andy H. F. Chow, Rui Sha, and Shuai Li. 2019. Centralised and decentralised signal timing optimisation approaches for network traffic control. *Transp. Res. Procedia* 38 (2019), 222–241.
[8] Andy H. F. Chow, Rui Sha, and Ying Li. 2019. Adaptive control strategies for urban network traffic via a decentralized approach with user-optimal routing. *IEEE Trans. Intell. Transp. Syst.* 21, 4 (2019), 1697–1704.
[9] Seung-Bae Cools, Carlos Gershenson, and Bart D'Hooghe. 2013. Self-organizing traffic lights: A realistic simulation. In *Advances in Applied Self-Organizing Systems*. Springer, 45–55.
[10] Christina Diakaki, Markos Papageorgiou, and Kostas Aboudolas. 2002. A multivariable regulator approach to traffic-responsive network-wide signal control. *Control Eng. Pract.* 10, 2 (2002), 183–195.
[11] Waqar Farooq, Muazzam Ali Khan, and Saad Rehman. 2016. A cluster based multicast routing protocol for autonomous unmanned military vehicles (AUMVs) communication in VANET. In *Proceedings of the International Conference on Computing, Electronic and Electrical Engineering (ICE Cube'16)*. IEEE, 42–48.
[12] GPIO (general-purpose input/output) pins. [n.d.]. Retrieved from https://www.raspberrypi.org/documentation/usage/gpio
[13] Akram Hakiri, Aniruddha Gokhale, and Pascal Berthou. 2019. Software-defined wireless mesh networking for reliable and real-time smart city cyber physical applications. In *Proceedings of the 27th International Conference on Real-Time Networks and Systems*. 165–175.
[14] P. B. Hunt, D. I. Robertson, R. D. Bretherton, and M. Cr Royle. 1982. The scoot on-line traffic signal optimisation technique. *Traffic Eng. Control* 23, 4 (1982).
[15] Shahid Hussain, Yun-Su Kim, Subhasis Thakur, and John G. Breslin. 2022. Optimization of waiting time for electric vehicles using a fuzzy inference system. *IEEE Transactions on Intelligent Transportation Systems* 23, 9 (2022), 15396–15407.
[16] Md Abdus Samad Kamal, Chee Pin Tan, Tomohisa Hayakawa, Shun-Ichi Azuma, and Jun-Ichi Imura. 2021. Control of vehicular traffic at an intersection using a cyber-physical multiagent framework. *IEEE Trans. Ind. Inform.* 17, 9 (2021), 6230–6240.
[17] Daniel Krajzewicz, Jakob Erdmann, Michael Behrisch, and Laura Bieker. 2012. Recent development and applications of sumo - simulation of urban mobility. *Int. J. Adv. Syst. Measure.* 5, 3&4 (Dec. 2012), 128–138. Retrieved from http://elib.dlr.de/80483/
[18] Werner Kraus, Felipe A. de Souza, Rodrigo C. Carlson, Markos Papageorgiou, Luciano D. Dantas, Eduardo Camponogara, Elias Kosmatopoulos, and Konstantinos Aboudolas. 2010. Cost effective real-time traffic signal control using the TUC strategy. *IEEE Intell. Transp. Syst. Mag.* 2, 4 (2010), 6–17.
[19] Li Kuang, Jianbo Zheng, Kemu Li, and Honghao Gao. 2021. Intelligent traffic signal control based on reinforcement learning with state reduction for smart cities. *ACM Trans. Internet Technol.* 21, 4 (2021), 1–24.
[20] Neetesh Kumar, Sarthak Mittal, Vaibhav Garg, and Neeraj Kumar. 2021. Deep reinforcement learning-based traffic light scheduling framework for SDN-enabled smart transportation system. *IEEE Transactions on Intelligent Transportation Systems* 23, 3 (2021), 2411–2421.
[21] Huibert Kwakernaak and Raphael Sivan. 1972. *Linear Optimal Control Systems*. Vol. 1. Wiley-Interscience, New York, NY.
[22] Imen Mahjri, Sébastien Faye, and Djamel Khadraoui. 2019. Impact and deployment of dynamic traffic light control strategies using a city-wide simulation scenario. In *Proceedings of the IEEE Intelligent Transportation Systems Conference (ITSC'19)*. 2213–2219. https://doi.org/10.1109/ITSC.2019.8916984
[23] Nick McKeown, Tom Anderson, Hari Balakrishnan, Guru Parulkar, Larry Peterson, Jennifer Rexford, Scott Shenker, and Jonathan Turner. 2008. OpenFlow: Enabling innovation in campus networks. *ACM SIGCOMM Comput. Commun. Rev.* 38, 2 (2008), 69–74.
[24] Pedro Mercader, Wasim Uwayid, and Jack Haddad. 2020. Max-pressure traffic controller based on travel times: An experimental analysis. *Transport. Res. C: Emerg. Technol.* 110 (2020), 275–290.
[25] Muhammad Zeshan Naseer. 2016. Modeling control traffic in distributed software defined networks. Degree Project in Electrical Engineering, KTH Royal Institute of Technology School of Electrical Engineering. Retrieved from http://www.diva-portal.org/smash/get/diva2:1038710/FULLTEXT01.pdf
[26] Mohammad Noaeen, Atharva Naik, Liana Goodman, Jared Crebo, Taimoor Abrar, Zahra Shakeri Hossein Abad, Ana L. C. Bazzan, and Behrouz Far. 2022. Reinforcement learning in urban network traffic signal control: A systematic literature review. *Exp. Syst. Appl.* (2022), 116830.
[27] Jiho Park, Tong Liu, Chieh Wang, Andy Berres, Joseph Severino, Juliette Ugirumurera, Airton G. Kohls, Hong Wang, Jibonananda Sanyal, and Zhong-Ping Jiang. 2022. Adaptive urban traffic signal control for multiple intersections: An LQR approach. In *Proceedings of the IEEE 25th International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2240–2245.
[28] Anuj Sachan, Nisha Singh Chauhan, and Neetesh Kumar. 2023. Congestion minimization using fog-deployed DRL-agent feedback enabled traffic light cooperative framework. In *Proceedings of the IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid'23)*. IEEE, 557–567.

[29] Anuj Sachan and Neetesh Kumar. 2022. SDN control-enabled and time-quantum-driven max-pressure approach for intersection management in smart city. *IEEE Syst. J.* (2022).
[30] Anuj Sachan and Neetesh Kumar. 2023. S-Edge: Heterogeneity-aware, light-weighted, and edge computing integrated adaptive traffic light control framework. *J. Supercomput.* (2023), 1–31.
[31] Pradip Kumar Sharma, Shailendra Rathore, Young-Sik Jeong, and Jong Hyuk Park. 2018. SoftEdgeNet: SDN-based energy-efficient distributed network architecture for edge computing. *IEEE Commun. Mag.* 56, 12 (2018), 104–111.
[32] Arthur G. Sims. 1979. The Sydney coordinated adaptive traffic system. In *Proceedings of the Engineering Foundation Conference on Research Directions in Computer Control of Urban Traffic Systems*.
[33] Robert F. Stengel. 1994. *Optimal Control and Estimation*. Courier Corp.
[34] Russ Tedrake. 2023. *Underactuated Robotics*. Retrieved from https://underactuated.csail.mit.edu
[35] Ishu Tomar, Indu Sreedevi, and Neeta Pandey. 2022. State-of-art review of traffic light synchronization for intelligent vehicles: Current status, challenges, and emerging trends. *Electronics* 11, 3 (2022), 465.
[36] K. Triantis, Sudipta Sarangi, D. Teodorović, and Laura Razzolini. 2011. Traffic congestion mitigation: Combining engineering and economic perspectives. *Transport. Plan. Technol.* 34, 7 (2011), 637–645.
[37] Pravin Varaiya. 2013. Max pressure control of a network of signalized intersections. *Trans. Res. Part C: Emer. Tech.* 36 (2013), 177–195.
[38] Hong Wang, Meixin Zhu, Wanshi Hong, Chieh Wang, Gang Tao, and Yinhai Wang. 2020. Optimizing signal timing control for large urban traffic networks using an adaptive linear quadratic regulator control strategy. *IEEE Transactions on Intelligent Transportation Systems* 23, 1 (2020), 333–343.
[39] Shivnath Yadav, Sunakshi Singh, and Vijay Kumar Chaurasiya. 2022. Traffic light control using RFID and deep reinforcement learning. In *AI and IoT for Smart City Applications*. Springer, 47–64.
[40] Weiqi Yu, Weichen Bai, Liang Qi, and Wenjing Luan. 2022. State-of-the-art review on traffic control strategies for emergency vehicles. *IEEE Access* 10 (2022), 109729–109742.
[41] Haibo Zhang, Lingyu Zhang, Zhonghe He, and Min Li. 2017. LQR-based perimeter control approach for urban road traffic networks. In *Proceedings of the 6th IEEE DDCLS*. 745–749.
[42] Zusheng Zhang, Xianmang He, and Huaqiang Yuan. 2019. An antiinterference traffic speed estimation system with wireless magnetic sensor network. *IEEE Trans. Ind. Inform.* 16, 4 (2019), 2458–2468.
[43] Qian Zou, Xingzhou Du, Yuezhen Liu, Hui Chen, Yibin Wang, and Jiangfan Yu. 2022. Dynamic path planning and motion control of microrobotic swarms for mobile target tracking. *IEEE Transactions on Automation Science and Engineering* 20, 4 (2022), 2454–2468.

Received 16 June 2023; revised 25 November 2023; accepted 11 January 2024